

Capture the Flag 4: The Student List

Goal: Build a scrollable student directory using Dart Lists and `ListView.builder`, replacing manually duplicated widgets with dynamically generated UI.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

Flag 0 — Create Your Feature Branch

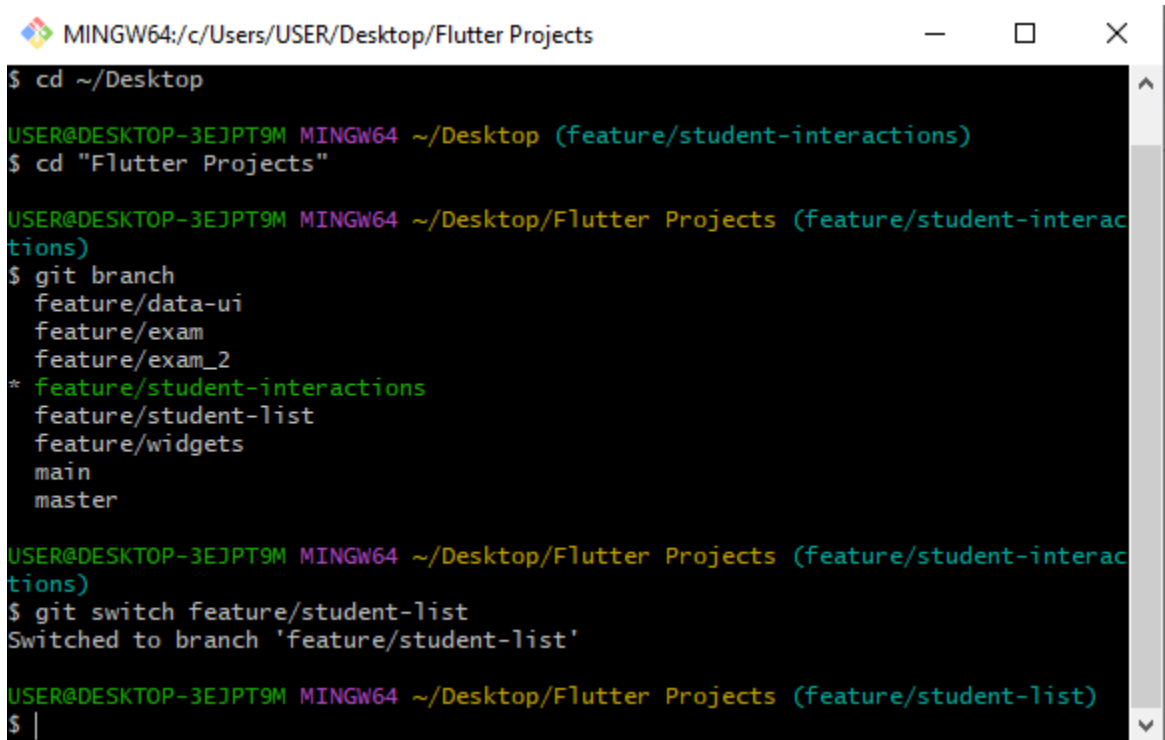
Objective: Create a feature branch before building your student list.

Create:

`feature/student-list`

Screenshot:

- Terminal showing your current branch



```
MINGW64:/c/Users/USER/Desktop/Flutter Projects
$ cd ~/Desktop
USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop (feature/student-interactions)
$ cd "Flutter Projects"
USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop/Flutter Projects (feature/student-interactions)
$ git branch
  feature/data-ui
  feature/exam
  feature/exam_2
* feature/student-interactions
  feature/student-list
  feature/widgets
  main
  master
USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop/Flutter Projects (feature/student-interactions)
$ git switch feature/student-list
Switched to branch 'feature/student-list'
USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop/Flutter Projects (feature/student-list)
$ |
```

🚩 Flag 1 — Build Your Student Data

Objective: Store multiple students inside a Dart `List`.

Research what a Dart `List` is and why it is useful when handling multiple pieces of similar data.

Instead of creating one student at a time with separate variables, create a collection that stores at least five students.

Each student should contain information such as:

- Image
- Name
- Course
- Year Level
- Age
- Hobby

Your data should exist separately from your UI widgets.

Screenshot:

Code:

- Your student list

```
21 List<Student> students = [  
22     Student(  
23         image: 'assets/zelon.jpg',  
24         name: 'Zelon Estimo',  
25         course: 'BSIT',  
26         yearLevel: '3rd Year',  
27         age: 21,  
28         hobby: 'Photography',  
29     ),  
30  
31     Student(  
32         image: 'assets/ryan.jpg',  
33         name: 'Ryan Porquiado',  
34         course: 'BSIT',  
35         yearLevel: '4th Year',  
36         age: 21,  
37         hobby: 'Drilling',  
38     ),  
39  
40     Student(  
41         image: 'assets/leon.jpg',  
42         name: 'Leonardo Cajés Jr.',  
43         course: 'BSTM',  
44         yearLevel: '4th Year',  
45         age: 21,  
46         hobby: 'Twerking',  
47     ),
```

```

49     Student(
50         image: 'assets/jackie.jpg',
51         name: 'Jacklyn Reyes',
52         course: 'BSTM',
53         yearLevel: '4th Year',
54         age: 22,
55         hobby: 'Fishing',
56     ),
57
58     Student(
59         image: 'assets/julie.jpg',
60         name: 'Julie Ann Malinao',
61         course: 'BSIT',
62         yearLevel: '4th Year',
63         age: 21,
64         hobby: 'Eating',
65     ),
66 ];

```

```

import package:flutter/material;

class Student {
  String image;
  String name;
  String course;
  String yearLevel;
  int age;
  String hobby;

  Student({
    required this.image,
    required this.name,
    required this.course,
    required this.yearLevel,
    required this.age,
    required this.hobby,
  });
}

```

- At least five student entries

Flag 2 — The Copy-Paste Trap

You already know how to create a Student Card.

You created five student cards manually.

You duplicated the same card design until all five students appear.

Stop.

Imagine having:

- 50 students.
- 500 students.
- 5,000 students.

Would copying the same widget over and over still be practical?

This flag is intentionally making you feel the problem before solving it.

Screenshot:

- Your five manually created student cards (do not change your past code yet)


```

105 |     ), // TextStyle
106 |   ), // Text
107 |
108 |   const SizedBox(height: 5),
109 |
110 |   Text(
111 |     student.course,
112 |     style: const TextStyle(fontSize: 16),
113 |   ), // Text
114 |
115 |   const SizedBox(height: 5),
116 |
117 |   Text(
118 |     "Age: ${student.age}",
119 |     style: const TextStyle(fontSize: 16),
120 |   ), // Text
121 |
122 |   const SizedBox(height: 5),
123 |
124 |   Text(
125 |     "Hobby: ${student.hobby}",
126 |     style: const TextStyle(fontSize: 16),
127 |   ), // Text
128 |   ],
129 | ), // Column
130 | ), // Expanded
131 | ],
132 | ), // Row
133 | ), // Padding
134 | ), // Card
135 | ],

```

🚩 Flag 3 — Discover **ListView**

Objective: Make your student directory scrollable using **ListView**.

Research Flutter's **ListView**.

Replace your previous layout with a scrolling layout so users can browse through all of your student cards naturally.

Your application should now allow continuous vertical scrolling.

Test

The user should be able to:

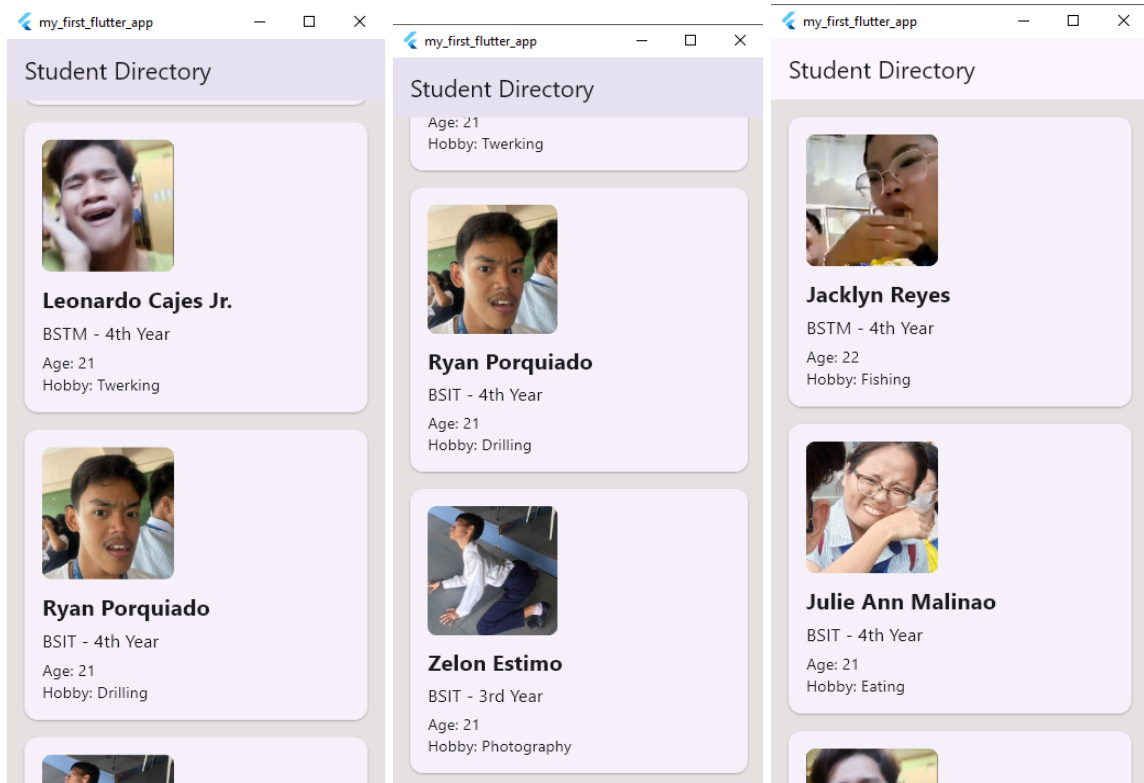
- Scroll from the first student
- Continue scrolling
- Reach the last student without resizing the screen

Screenshot:

- `ListView` in your code

```
body: ListView(  
  children: [  
    // ...  
  ],  
)
```

- Top of the application
- Bottom portion showing later students



🚩 Flag 4 — Unlock `ListView.builder`

Objective: Generate student cards automatically using `ListView.builder`.

Research `ListView.builder`.

Instead of writing five separate Student Cards, generate them automatically from your student list.

Your UI should now create one card for each student in your list.

Test

Add one new student to your list.

Without creating another card manually, your application should display the new student automatically.

Screenshot:

- `ListView.builder` in your code

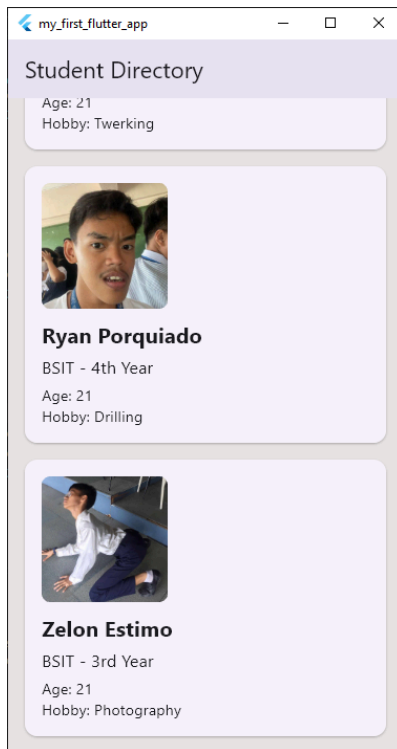
```
body: ListView.builder(  
  padding: const EdgeInsets.all(8),  
  itemCount: students.length,  
  itemBuilder: (context, index) {  
    return studentCard(students[index]);  
  },  
), // ListView.builder
```

- Student list

```
21 List<Student> students = [  
22   Student(  
23     image: 'assets/zelon.jpg',  
24     name: 'Zelon Estimo',  
25     course: 'BSIT',  
26     yearLevel: '3rd Year',  
27     age: 21,  
28     hobby: 'Photography',  
29   ),  
30  
31   Student(  
32     image: 'assets/ryan.jpg',  
33     name: 'Ryan Porquiado',  
34     course: 'BSIT',  
35     yearLevel: '4th Year',  
36     age: 21,  
37     hobby: 'Drilling',  
38   ),  
39  
40   Student(  
41     image: 'assets/leon.jpg',  
42     name: 'Leonardo Cajés Jr.',  
43     course: 'BSTM',  
44     yearLevel: '4th Year',  
45     age: 21,  
46     hobby: 'Twerking',  
47   ),  
48 ];
```

```
49   Student(  
50     image: 'assets/jackie.jpg',  
51     name: 'Jacklyn Reyes',  
52     course: 'BSTM',  
53     yearLevel: '4th Year',  
54     age: 22,  
55     hobby: 'Fishing',  
56   ),  
57  
58   Student(  
59     image: 'assets/julie.jpg',  
60     name: 'Julie Ann Malinao',  
61     course: 'BSIT',  
62     yearLevel: '4th Year',  
63     age: 21,  
64     hobby: 'Eating',  
65   ),  
66 ];
```

- Newly generated student appearing in the app



🚩 Flag 5 — Every Student Deserves an ID

Objective: Expand your student records with additional information.

Real student directories contain more than just names.

Add at least three new fields to every student.

Examples:

- Student ID
- Email
- Favorite Subject

Update your Student Card so the new information appears consistently across every generated card.

Screenshot:

- Updated student data

```

27 List<Student> students = [
28     Student(
29         image: 'assets/zelon.jpg',
30         name: 'Zelon Estimo',
31         course: 'BSIT',
32         yearLevel: '3rd Year',
33         age: 21,
34         hobby: 'Photography',
35         studentId: '2980',
36         email: 'estimozelon@gmail.com',
37         favoriteSubject: 'UI/UX Designing',
38     ),
39     Student(
40         image: 'assets/ryan.jpg',
41         name: 'Ryan Porquiado',
42         course: 'BSIT',
43         yearLevel: '4th Year',
44         age: 21,
45         hobby: 'Drilling',
46         studentId: '2986',
47         email: 'ryanporquiado@gmail.com',
48         favoriteSubject: 'Recess',
49     ),
50 ],
51

```

```

Student(
    image: 'assets/leon.jpg',
    name: 'Leonardo Cajés Jr.',
    course: 'BSTM',
    yearLevel: '4th Year',
    age: 21,
    hobby: 'Twerking',
    studentId: '2988',
    email: 'leonardocajes@gmail.com',
    favoriteSubject: 'Research',
),

Student(
    image: 'assets/jackie.jpg',
    name: 'Jacklyn Reyes',
    course: 'BSTM',
    yearLevel: '4th Year',
    age: 22,
    hobby: 'Fishing',
    studentId: '6374',
    email: 'jacklynreyes@gmail.com',
    favoriteSubject: 'Traveling & Tour',
),

```

```

3  class Student {
4      String image;
5      String name;
6      String course;
7      String yearLevel;
8      int age;
9      String hobby;
10     String studentId;
11     String email;
12     String favoriteSubject;
13
14     Student({
15         required this.image,
16         required this.name,
17         required this.course,
18         required this.yearLevel,
19         required this.age,
20         required this.hobby,
21         required this.studentId,
22         required this.email,
23         required this.favoriteSubject,
24     });
25 }
26

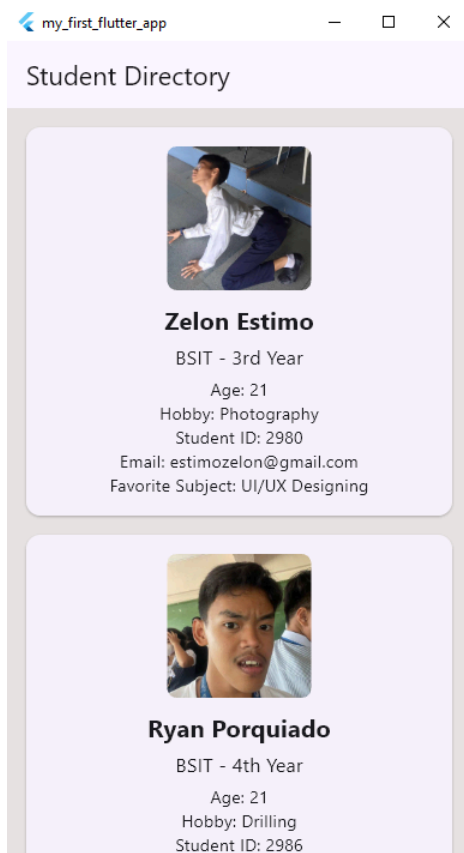
```

```

Student(
    image: 'assets/julie.jpg',
    name: 'Julie Ann Malinao',
    course: 'BSIT',
    yearLevel: '4th Year',
    age: 21,
    hobby: 'Eating',
    studentId: '3234',
    email: 'julieannmalinao@gmail.com',
    favoriteSubject: 'Fundamentals of Database System',
),
];

```

- Student Card displaying the additional fields



🚩 Flag 6 — Empty Classroom

What happens when there are no students?

Objective: Handle an empty `List` gracefully.

Imagine your app just connected to a database.

The database returns:

```
[]
```

No students.

What should the user see?

Instead of displaying a broken or confusing screen, research how Flutter can display a different widget when a condition is met.

Create an empty state that clearly communicates that there are currently no students available.

Examples:

- "No students found."
- "Student list is empty."

Test

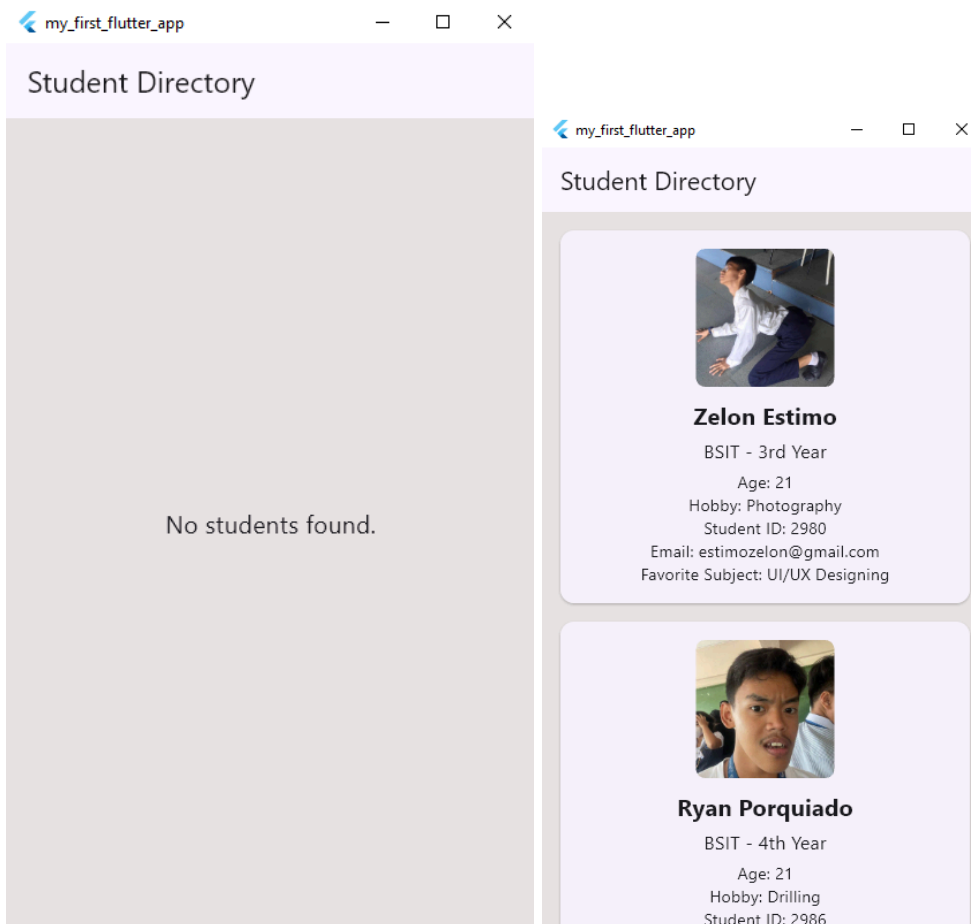
- Display your student list normally.
- Then temporarily make your list empty.
- Your application should display the empty state instead of student cards.

Screenshot:

- Empty student list

```
27 List<Student> students = [];
```

- Empty-state UI



- Application still functioning correctly

Flag 7 — Sort the Roll Call

Objective: Change the order of your student list.

Class lists are rarely displayed randomly.

Research how Dart can rearrange items inside a [List](#).

Sort your students alphabetically by name.

Test

Before sorting:

- Maria
- John
- Aaron
- Sophia

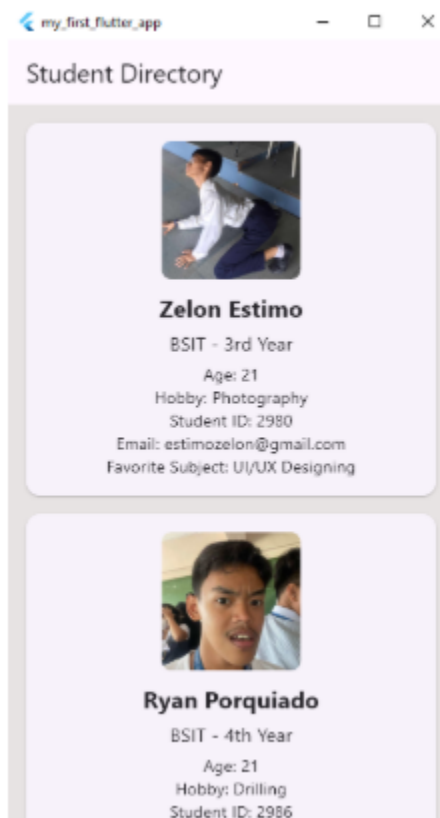
After sorting:

- Aaron
- John
- Maria
- Sophia

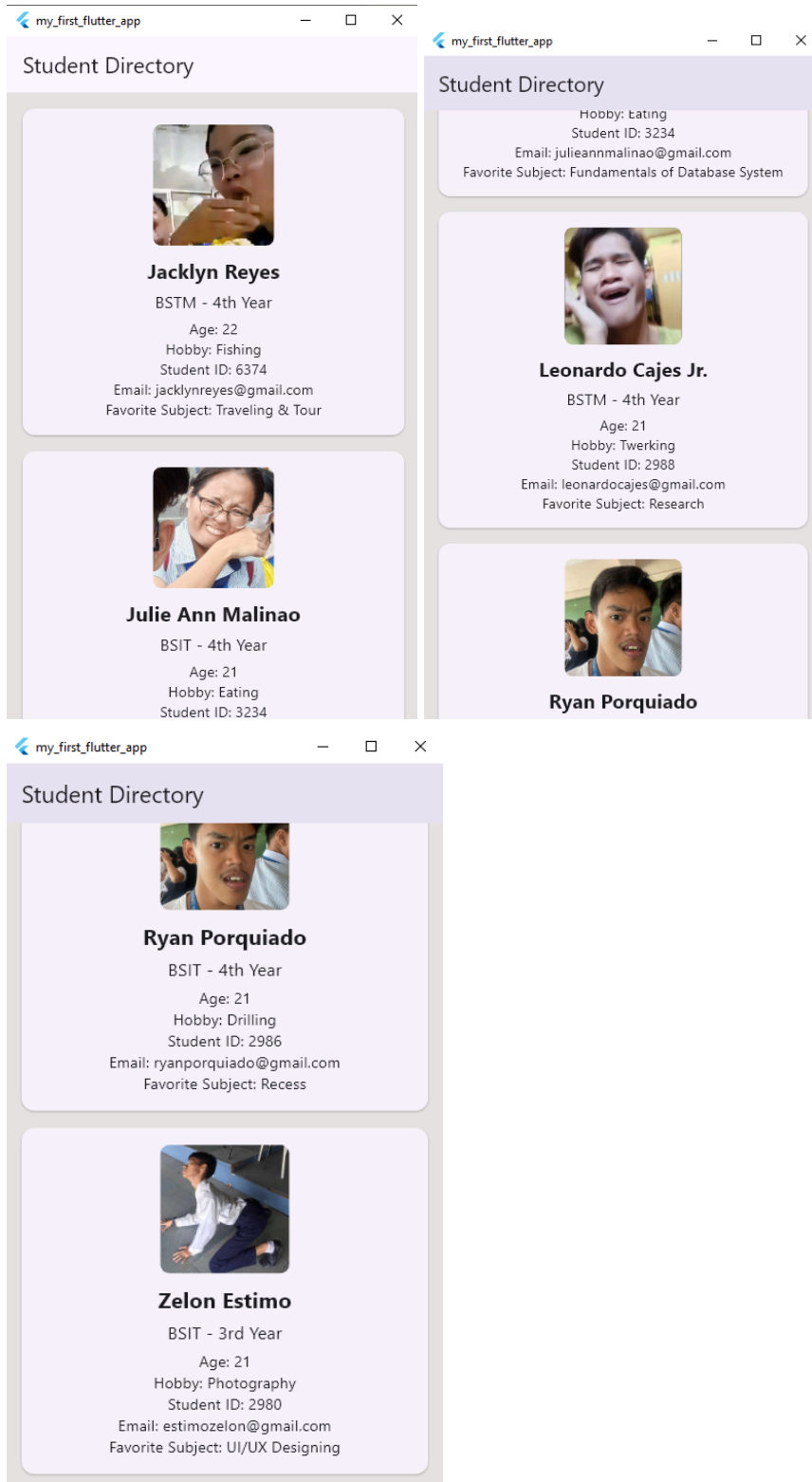
Your UI should immediately reflect the new order.

Screenshot:

- Student list before or after sorting



- Application displaying the sorted order



🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Student List CTF.

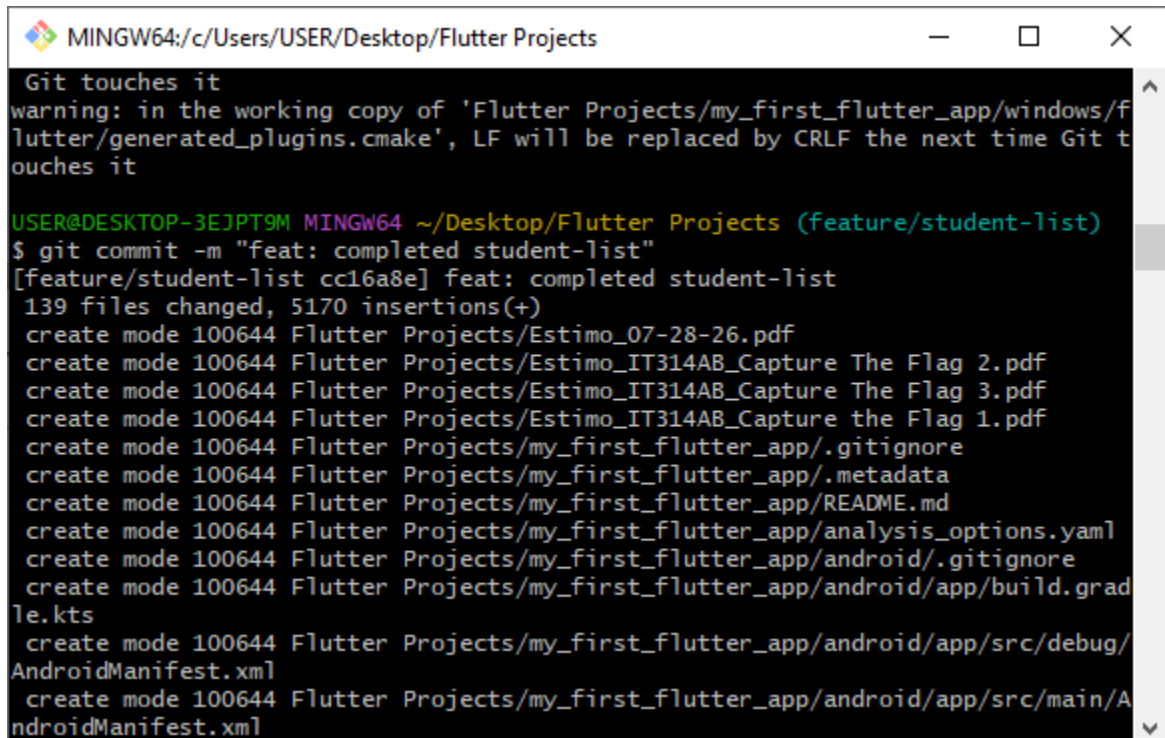
Use an appropriate commit message following proper Git standards.

Branch:

`feature/student-list`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit



```
MINGW64:/c/Users/USER/Desktop/Flutter Projects
Git touches it
warning: in the working copy of 'Flutter Projects/my_first_flutter_app/windows/flutter/generated_plugins.cmake', LF will be replaced by CRLF the next time Git touches it

USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop/Flutter Projects (feature/student-list)
$ git commit -m "feat: completed student-list"
[feature/student-list cc16a8e] feat: completed student-list
139 files changed, 5170 insertions(+)
 create mode 100644 Flutter Projects/Estimo_07-28-26.pdf
 create mode 100644 Flutter Projects/Estimo_IT314AB_Capture The Flag 2.pdf
 create mode 100644 Flutter Projects/Estimo_IT314AB_Capture The Flag 3.pdf
 create mode 100644 Flutter Projects/Estimo_IT314AB_Capture the Flag 1.pdf
 create mode 100644 Flutter Projects/my_first_flutter_app/.gitignore
 create mode 100644 Flutter Projects/my_first_flutter_app/.metadata
 create mode 100644 Flutter Projects/my_first_flutter_app/README.md
 create mode 100644 Flutter Projects/my_first_flutter_app/analysis_options.yaml
 create mode 100644 Flutter Projects/my_first_flutter_app/android/.gitignore
 create mode 100644 Flutter Projects/my_first_flutter_app/android/app/build.gradle
 create mode 100644 Flutter Projects/my_first_flutter_app/android/app/src/debug/AndroidManifest.xml
 create mode 100644 Flutter Projects/my_first_flutter_app/android/app/src/main/AndroidManifest.xml
```